Отчёт по лабораторной работе №8

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **6**

2 Предварительные сведения **7**

3 Выполнение лабораторной работы **8**

3.1 Линейное программирование 8
3.2 Векторизованные ограничения и целевая функция оптимизации . 12
3.3 Оптимизация рациона питания 15
3.4 Путешествие по миру 21
3.5 Портфельные инвестиции 24
3.6 Восстановление изображения 33
3.7 Задания для самостоятельного выполнения 37

4 Выводы **41**

Список иллюстраций

3.1 Формула общей задачи линейного программирования 8
3.2 Ограничения в форме неравенств 8
3.3 Ограничения 9
3.4 Подключение пакетов 10
3.5 Определение объекта модели с именем model 10
3.6 Определение переменных x,y и граничных условий для них 11
3.7 Определение ограничений модели 11
3.8 Определение целевой функции 11
3.9 Вызов функции оптимизации 11
3.10 Определение причины завершения работы оптимизатора 12
3.11 Результат 12
3.12 Подключение пакетов 13
3.13 Определение объекта модели с именем vector_model 13
3.14 Определение начальных данных 14
3.15 Определение вектора переменных 14
3.16 Определение ограничений модели 14
3.17 Определение целевой функции 15
3.18 Вызов функции оптимизации 15
3.19 Определение причины завершения работы оптимизатора 15
3.20 Результат 15
3.21 Подключение пакетов 16
3.22 Создание контейнера JuMP для хранения инфо об ограничениях . 17
3.23 Массив данных с наименованиями продуктов 17
3.24 Массив стоимости продуктов 17
3.25 Массив данных о содержании калорий, белков, жиров и соли . . . 18
3.26 Определение объекта модели с именем model 18
3.27 Определим массив 19
3.28 Определение переменных 19
3.29 Определение целевой функции 19
3.30 Определение ограничений модели 20
3.31 Вызов функции оптимизации 20
3.32 Результат 20
3.33 Скачиваем данные с ресурса на git 21
3.34 Подключение пакетов 21
3.35 Считывание данных 22
3.36 Подключение пакетов 22
3.37 Задаём переменные 23

3.38 Определение объекта модели с именем model 23
3.39 Переменные, ограничения и целевая функция 23
3.40 Вызов функции оптимизации 23
3.41 Просмотр результата 24
3.42 Подключение необходимых пакетов 25
3.43 Считываем данные и размещаем их во фрейм 26
3.44 Построение графика 27
3.45 Данные о ценах на акции размещаем в матрице 28
3.46 Вычисление матрицы доходности за период времени 28
3.47 Матрица рисков 29
3.48 Проверка положительной определённости матрицы рисков 29
3.49 Доход от каждой из компаний 29
3.50 Вектор инвестиций 29
3.51 Объект модели 30
3.52 Находим решение 31
3.53 Результат 32
3.54 Проверка выполнения условия 32
3.55 Проверка выполнения условия на уровень доходности от 2% 32
3.56 Перевод процентных значений компонент вектора инвестиций в фактические денежные единицы 32
3.57 Подключение необходимых пакетов 33
3.58 Считывание исходного изображения 34
3.59 Преобразование изображения в оттенки серого и порча пикселей 35
3.60 Матрица цветов 36
3.61 Формирование матрицы с минимизированием нормы ядра матрицы 37
3.62 Находим решение 37
3.63 Задание №1 38
3.64 Задание №2 39
3.65 Задание №3 40
3.66 Задание №3 40

Список таблиц

1 Цель работы

Основная цель работа — освоить пакеты Julia для решения задач оптимизации.

2 Предварительные сведения

Под оптимизацией в математике и информатике понимается решение задачи нахождения экстремума (минимума или максимума) целевой функции в некоторой области конечномерного векторного пространства, ограниченной набором линейных и/или нелинейных равенств и/или неравенств. Оптимизационной задачей называется задача определения наилучших с точки зрения структуры или значений параметров объектов.

3 Выполнение лабораторной работы

3.1 Линейное программирование

Линейное программирование рассматривает решения экстремальных задач на множествах □-мерного векторного пространства, задаваемых системами линейных уравнений и неравенств. Общей (стандартной) задачей линейного программирования называется задача нахождения минимума линейной целевой функции вида:

$$\sum_{j=1}^{n} a_{ij}x_j \geqslant b_i, \quad i = 1, 2, \dots, m, \quad x_j \geqslant 0, \quad j = 1, 2, \dots, n.$$

Рис. 3.1: Формула общей задачи линейного программирования

где $\vec{c}$ — некоторые коэффициенты, $\vec{x} \in \mathbb{R}^n$

Основной задачей линейного программирования называется задача, в которой есть ограничения в форме неравенств:

$$6x + 8y \geqslant 100, \quad 7x + 12y \geqslant 120, \quad x \geqslant 0, \quad y \geqslant 0.$$

Рис. 3.2: Ограничения в форме неравенств

Задачи линейного программирования со смешанными ограничениями, такими как равенства и неравенства, с наличием переменных, свободных от ограничений, могут быть сведены к эквивалентным с тем же множеством решений путём

замены переменных и замены равенств на пару неравенств. В Julia есть несколько средств, предназначенных для решения оптимизационных задач. Одним из таких средств является JuMP (https://jump.dev/) — язык моделирования и вспомогательные пакеты для формулирования и решения задач математической оптимизации в Julia. JuMP включает пакет Convex.jl (https://jump.dev/Convex.jl/stable/), позволяющий описать задачу оптимизации, используя естественный математический синтаксис, и решать её с помощью одного из решателей (COSMO, ECOS, SCS, GLPK, MathOptInterface). Предположим, что требуется решить следующую задачу линейного программирования:

$12x + 20y \to min$

при заданных ограничениях:

$$A = \begin{pmatrix} 1 & 1 & 9 & 5 \\ 3 & 5 & 0 & 8 \\ 2 & 0 & 6 & 13 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 7 \\ 3 \\ 5 \end{pmatrix}, \quad \vec{c} = \begin{pmatrix} 1 \\ 3 \\ 5 \\ 2 \end{pmatrix}.$$

Рис. 3.3: Ограничения

Воспользуемся JuMP и решателем линейного и смешанного целочисленного программирования GLPK:

```
[*]: import Pkg
     Pkg.add("JuMP")
     Pkg.add("GLPK")
     using JuMP
     using GLPK
```

```
    Updating registry at `C:\Users\eldar\.julia\registries\General.toml`
   Resolving package versions...
   Installed CodecBzip2 ──────── v0.8.5
   Installed JuMP ─────────────── v1.30.0
   Installed MathOptInterface ─ v1.49.0
    Updating `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
  [4076af6c] + JuMP v1.30.0
    Updating `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
  [523fee87] + CodecBzip2 v0.8.5
  [4076af6c] + JuMP v1.30.0
  [b8f27783] + MathOptInterface v1.49.0
```

Рис. 3.4: Подключение пакетов

Объект модели (контейнер для переменных, ограничений, параметров решателя и т. д.) в JuMP создаётся при помощи функции Model(), в которой в качестве аргумента указывается оптимизатор (решатель):

```
[3]: # Определение объекта модели с именем model:
     model = Model(GLPK.Optimizer)
```

```
[3]: A JuMP Model
     ├ solver: GLPK
     ├ objective_sense: FEASIBILITY_SENSE
     ├ num_variables: 0
     ├ num_constraints: 0
     └ Names registered in the model: none
```

Рис. 3.5: Определение объекта модели с именем model

Переменные задаются с помощью конструкции [**variable?**] (имя объекта модели, имя и привязка переменной, тип переменной). Здесь же задаются граничные условия на переменные (если тип переменной не определён, он считается действительным):

```
[4]: # Определение переменных x, y и граничных условий для них:
     @variable(model, x >= 0)
     @variable(model, y >= 0)
```

[4]: y

Рис. 3.6: Определение переменных x,y и граничных условий для них

В качестве первого аргумента указан объект модели model, затем переменные □ и □, связанные с этой моделью (причём указанные переменные не могут использоваться в другой модели). Ограничения модели задаются с помощью конструкции [**constraint?**] (имя объекта модели, ограничение):

```
[5]: # Определение ограничений модели:
     @constraint(model, 6x + 8y >= 100)
     @constraint(model, 7x + 12y >= 120)
```

[5]: $7x + 12y \geq 120$

Рис. 3.7: Определение ограничений модели

Далее следует задать собственно целевую функцию с помощью конструкции [**objective?**] (имя объекта модели, Min или Max, функция для оптимизации):

```
[6]: # Определение целевой функции:
     @objective(model, Min, 12x + 20y)
```

[6]: $12x + 20y$

Рис. 3.8: Определение целевой функции

Для решения задачи оптимизации необходимо вызывать функцию оптимизации:

```
[7]: # Вызов функции оптимизации:
     optimize!(model)
```

Рис. 3.9: Вызов функции оптимизации

Следует проверять причину прекращения работы оптимизатора, используя конструкцию termination_status(Объект модели):

```
[8]: # Определение причины завершения работы оптимизатора:
     termination_status(model)

[8]: OPTIMAL::TerminationStatusCode = 1
```

Рис. 3.10: Определение причины завершения работы оптимизатора

Процесс решения мог быть прекращён по ряду причин. Во-первых, решатель мог найти оптимальное решение или доказать, что проблема невозможна. Однако он также мог столкнуться с численными трудностями или прерваться из-за таких настроек, как ограничение по времени. Если возвращено значение OPTIMAL, найдено оптимальное решение. Наконец, можно посмотреть собственно результат решения оптимизационной задачи:

```
[10]: # Демонстрация первичных результирующих значений переменных x и y:
      @show value(x);
      @show value(y);
      # Демонстрация результата оптимизации:
      @show objective_value(model);

      value(x) = 14.999999999999993
      value(y) = 1.2500000000000047
      objective_value(model) = 205.0
```

Рис. 3.11: Результат

3.2 Векторизованные ограничения и целевая функция оптимизации

Часто бывает полезно создавать коллекции переменных JuMP внутри более сложных структур данных. Можно добавить ограничения и цель в JuMP, используя векторизованную линейную алгебру. Предположим, что требуется решить следующую задачу:

$\vec{c}^T\vec{x} \to \min$

при заданных ограничениях:

$A\vec{x} = \vec{b}, \quad \vec{x} \geq 0, \quad \vec{x} \in \mathbb{R}$

где

$$A = \begin{pmatrix} 1 & 1 & 9 & 5 \\ 3 & 5 & 0 & 8 \\ 2 & 0 & 6 & 13 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 7 \\ 3 \\ 5 \end{pmatrix}, \quad \vec{c} = \begin{pmatrix} 1 \\ 3 \\ 5 \\ 2 \end{pmatrix}.$$

Воспользуемся JuMP и решателем линейного и смешанного целочисленного программирования GLPK:

```
# Подключение пакетов:
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK
```

```
   Resolving package versions...
     Project No packages added to or remo
    Manifest No packages added to or remo
Precompiling packages...
```

Рис. 3.12: Подключение пакетов

Определим объект модели:

```
[12]: # Определение объекта модели с именем vector_model:
      vector_model = Model(GLPK.Optimizer)
```

```
[12]: A JuMP Model
      ├ solver: GLPK
      ├ objective_sense: FEASIBILITY_SENSE
      ├ num_variables: 0
      ├ num_constraints: 0
      └ Names registered in the model: none
```

Рис. 3.13: Определение объекта модели с именем vector_model

Зададим исходные значения матрицы A и векторов $\vec{b},\vec{c}$:

```
[13]: # Определение начальных данных:
      A= [ 1 1 9 5;
      3 5 0 8;
      2 0 6 13]
      b = [7; 3; 5]
      c = [1; 3; 5; 2]
```

```
[13]: 4-element Vector{Int64}:
       1
       3
       5
       2
```

Рис. 3.14: Определение начальных данных

Далее зададим массив переменных для компонент вектора $\vec{x}$:

```
[14]: # Определение вектора переменных:
      @variable(vector_model, x[1:4] >= 0)
```

```
[14]: 4-element Vector{VariableRef}:
       x[1]
       x[2]
       x[3]
       x[4]
```

Рис. 3.15: Определение вектора переменных

Затем зададим ограничения в соответствии с условиями модели:

```
[15]: # Определение ограничений модели:
      @constraint(vector_model, A * x .== b)
```

```
[15]: 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
       x[1] + x[2] + 9 x[3] + 5 x[4] == 7
       3 x[1] + 5 x[2] + 8 x[4] == 3
       2 x[1] + 6 x[3] + 13 x[4] == 5
```

Рис. 3.16: Определение ограничений модели

Далее зададим целевую функцию:

```
[16]: # Определение целевой функции:
      @objective(vector_model, Min, c' * x)
```

[16]: $x_1 + 3x_2 + 5x_3 + 2x_4$

Рис. 3.17: Определение целевой функции

Наконец, для решения задачи оптимизации вызовем функцию оптимизации:

```
[17]: # Вызов функции оптимизации:
      optimize!(vector_model)
```

Рис. 3.18: Вызов функции оптимизации

Проверим, что найдено оптимальное решение:

```
[18]: # Определение причины завершения работы оптимизатора:
      termination_status(vector_model)

[18]: OPTIMAL::TerminationStatusCode = 1
```

Рис. 3.19: Определение причины завершения работы оптимизатора

Посмотрим результат решения оптимизационной задачи:

```
[19]: # Демонстрация результата оптимизации:
      @show objective_value(vector_model);

      objective_value(vector_model) = 4.9230769230769225
```

Рис. 3.20: Результат

3.3 Оптимизация рациона питания

В некоторых задачах требуется использование массивов, в которых индексы не являются целыми диапазонами с отсчётом от единицы. Например, требуется использовать переменную, индексируемую по названию продукта или местоположению. Тогда необходимо в качестве индекса использовать произвольный

вектор. В Julia это можно реализовать с помощью DenseAxisArrays. Рассмотрим применение DenseAxisArrays на примере решения задачи оптимизации рациона питании в заведении быстрого питания при условии, что задано ограничение на количество потребляемых калорий (1800–2200), белков (□ 91), жиров (0–65) и соли (0–1779), а также перечень определённых продуктов питания с указанием их стоимости — гамбургер (2.49 ден.ед.), курица (2.89 ден.ед.), сосиска в тесте (1.50 ден.ед.), жареный картофель (1.89 ден.ед.), макароны (2.09 ден.ед.), пицца (1.99 ден.ед.), салат (2.49 ден.ед.), молочный коктейль (0.89 ден.ед.), мороженное (1.59 ден.ед.). Также известно содержание калорий, белков, жиров и соли в указанных продуктах.

Воспользуемся JuMP и решателем линейного и смешанного целочисленного программирования GLPK :

```
[*]: # Подключение пакетов:
     import Pkg
     Pkg.add("JuMP")
     Pkg.add("GLPK")
     using JuMP
     using GLPK
```

```
   Resolving package versions...
     Project No packages added to or remo
    Manifest No packages added to or remo
Precompiling packages...
```

Рис. 3.21: Подключение пакетов

Создадим контейнер JuMP для хранения информации об ограничениях (минимальное, максимальное) на количество потребляемых калорий, белков, жиров и соли:

```
[21]: # Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray(
    [1800 2200;
    91 Inf;
    0 65;
    0 1779],
    ["calories", "protein", "fat", "sodium"],
    ["min", "max"])
```

```
[21]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, ["calories", "protein", "fat", "sodium"]
    Dimension 2, ["min", "max"]
And data, a 4×2 Matrix{Float64}:
 1800.0  2200.0
   91.0     Inf
    0.0    65.0
    0.0  1779.0
```

Рис. 3.22: Создание контейнера JuMP для хранения инфо об ограничениях

Введём массив данных с наименованиями продуктов:

```
[22]: # массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza","salad", "milk", "ice cream"]
```

```
[22]: 9-element Vector{String}:
 "hamburger"
 "chicken"
 "hot dog"
 "fries"
 "macaroni"
 "pizza"
 "salad"
 "milk"
 "ice cream"
```

Рис. 3.23: Массив данных с наименованиями продуктов

Введём данные о стоимости продуктов:

```
[23]: # Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray(
[2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
foods)
```

```
[23]: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.49
 2.89
 1.5
 1.89
 2.09
 1.99
 2.49
 0.89
 1.59
```

Рис. 3.24: Массив стоимости продуктов

Введём сведения о содержании калорий, белков, жиров и соли в продуктах питания:

```
[24]: # Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray(
[410 24 26 730;
420 32 10 1190;
560 20 32 1800;
380 4 19 270;
320 12 10 930;
320 15 12 820;
320 31 12 1230;
100 8 2.5 125;
330 8 10 180],
foods,
["calories", "protein", "fat", "sodium"])
```

```
[24]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
    Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9×4 Matrix{Float64}:
 410.0  24.0  26.0   730.0
 420.0  32.0  10.0  1190.0
 560.0  20.0  32.0  1800.0
 380.0   4.0  19.0   270.0
 320.0  12.0  10.0   930.0
 320.0  15.0  12.0   820.0
 320.0  31.0  12.0  1230.0
 100.0   8.0   2.5   125.0
 330.0   8.0  10.0   180.0
```

Рис. 3.25: Массив данных о содержании калорий, белков, жиров и соли

Определим объект модели:

```
[25]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)
```

```
[25]: A JuMP Model
      ├ solver: GLPK
      ├ objective_sense: FEASIBILITY_SENSE
      ├ num_variables: 0
      ├ num_constraints: 0
      └ Names registered in the model: none
```

Рис. 3.26: Определение объекта модели с именем model

Определим массив:

```
[26]: # Определим массив:
      categories = ["calories", "protein", "fat", "sodium"]
```

```
[26]: 4-element Vector{String}:
       "calories"
       "protein"
       "fat"
       "sodium"
```

Рис. 3.27: Определим массив

Далее зададим переменные:

```
[27]: # Определение переменных:
      @variables(model, begin
      category_data[c, "min"] <= nutrition[c = categories] <= category_data[c, "max"]
      # Сколько покупать продуктов:
      buy[foods] >= 0
      end)
```

```
[27]: (1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
          Dimension 1, ["calories", "protein", "fat", "sodium"]
      And data, a 4-element Vector{VariableRef}:
       nutrition[calories]
       nutrition[protein]
       nutrition[fat]
       nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
          Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
      And data, a 9-element Vector{VariableRef}:
       buy[hamburger]
       buy[chicken]
       buy[hot dog]
       buy[fries]
       buy[macaroni]
       buy[pizza]
       buy[salad]
       buy[milk]
       buy[ice cream])
```

Рис. 3.28: Определение переменных

Задаём целевую функцию минимизации цены:

```
[28]: # Определение целевой функции:
      @objective(model, Min, sum(cost[f] * buy[f] for f in foods))
```

[28]: $2.49buy_{hamburger} + 2.89buy_{chicken} + 1.5buy_{hotdog} + 1.89buy_{fries} + 2.09buy_{macaroni} + 1.99buy_{pizza} + 2.49buy_{salad} + 0.89buy_{milk} + 1.59buy_{icecream}$

Рис. 3.29: Определение целевой функции

Затем зададим ограничения в соответствии с условиями модели:

```
[30]: # Определение ограничений модели:
@constraint(model, [c in categories],
sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c]
)
```

```
[30]: 1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarSh
ape},1,...} with index sets:
    Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarSh
ape}}:
 -nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 320 buy[pizza] + 320 buy[salad] + 100 buy[milk] + 330 b
uy[ice cream] == 0
 -nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 8 buy[milk] + 8 buy[ice cream]
== 0
 -nutrition[fat] + 26 buy[hamburger] + 10 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 12 buy[salad] + 2.5 buy[milk] + 10 buy[ice cream]
== 0
 -nutrition[sodium] + 730 buy[hamburger] + 1190 buy[chicken] + 1800 buy[hot dog] + 270 buy[fries] + 930 buy[macaroni] + 820 buy[pizza] + 1230 buy[salad] + 125 buy[milk] + 180
buy[ice cream] == 0
```

Рис. 3.30: Определение ограничений модели

Наконец, для решения задачи оптимизации вызовем функцию оптимизации:

```
[31]: # Вызов функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)
```

```
[31]: OPTIMAL::TerminationStatusCode = 1
```

Рис. 3.31: Вызов функции оптимизации

Для просмотра результата решения модно вывести значение переменной buy:

```
[32]: hcat(buy.data,JuMP.value.(buy.data))
```

```
[32]: 9×2 Matrix{AffExpr}:
 buy[hamburger]  0.6045138888888888
 buy[chicken]    0
 buy[hot dog]    0
 buy[fries]      0
 buy[macaroni]   0
 buy[pizza]      0
 buy[salad]      0
 buy[milk]       6.9701388888888935
 buy[ice cream]  2.591319444444441
```

Рис. 3.32: Результат

В результате оптимальным по цене и пищевой ценности будет предложено купить гамбургер, молочный коктейль и мороженное.

3.4 Путешествие по миру

Рассмотрим решение задачи определения оптимального числа паспортов, требующихся, чтобы объехать весь мир. При этом будем учитывать сведения о паспортах и ограничениях, указанных на ресурсе https://www.passportindex.org/. В табличной форме данные можно получить с ресурса https://github.com/ilyankou/passport-index-dataset. Для решения задачи представляют интерес данные, указанные в файле passport-index-matrix.csv, в котором в первым столбце (Passport) указана страна отбытия (= от), в остальных столбцах указаны страны назначения (= до), на пересечении строк и столбцов указаны условия по наличию или отсутствию визы или другие ограничения

Итак, попробуем решить задачу средствами Julia. Сначала требуется скачать файл с данными. Например для ОС типа Linux можно воспользоваться стандартным вызовом команды git с соответствующими параметрами:

```
[33]: # Скачиваем данные с ресурса на git:
      ;git clone https://github.com/ilyankou/passport-index-dataset.git

Cloning into 'passport-index-dataset'...
```

Рис. 3.33: Скачиваем данные с ресурса на git

Затем требуется подключить пакеты для обработки табличных файлов:

```
]: # Подключение пакетов:
   import Pkg
   Pkg.add("DelimitedFiles")
   Pkg.add("CSV")
   using DelimitedFiles
   using CSV

      Resolving package versions...
       Updating `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
    [8bb1440f] + DelimitedFiles v1.9.1
       Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
   Precompiling packages...
```

Рис. 3.34: Подключение пакетов

Можем считать данные из имеющегося файла:

```
[35]: # Считывание данных:
passportdata = readdlm(joinpath("passport-index-dataset","passport-index-matrix.csv"),',')
```

```
[35]: 200×200 Matrix{Any}:
 "Passport"                 "Albania"  …    "Afghanistan"
 "Afghanistan"              "e-visa"      -1
 "Albania"                 -1                "visa required"
 "Algeria"                  "e-visa"         "visa required"
 "Andorra"                 90                "visa required"
 "Angola"                   "e-visa"   …     "visa required"
 "Antigua and Barbuda"     90                "visa required"
 "Argentina"               90                "visa required"
 "Armenia"                 90                "visa required"
 "Australia"               90                "visa required"
 "Austria"                 90           …    "visa required"
 "Azerbaijan"              90                "visa required"
 "Bahamas"                 90                "visa required"
 ⋮                                      ⋱
 "United Arab Emirates"    90                "visa required"
 "United Kingdom"          90                "visa required"
 "United States"          360           …    "visa required"
 "Uruguay"                 90                "visa required"
 "Uzbekistan"               "e-visa"         "visa required"
 "Vanuatu"                  "e-visa"         "visa required"
 "Vatican"                 90                "visa required"
 "Venezuela"               90           …    "visa required"
 "Vietnam"                  "e-visa"         "visa required"
 "Yemen"                    "e-visa"         "visa required"
 "Zambia"                   "e-visa"         "visa required"
 "Zimbabwe"                 "e-visa"         "visa required"
```

Рис. 3.35: Считывание данных

Для решения задачи используем возможности пакета JuMP и решателя линейного и смешанного целочисленного программирования GLPK:

```
# Подключение пакетов:
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK
```

```
   Resolving package versions...
     Project No packages added to or remo
    Manifest No packages added to or remo
```

Рис. 3.36: Подключение пакетов

Далее просматриваем файл, задаём переменную для подсчёта числа паспортов,

задаём переменную vf, в которую заносим сведения при отсутствии необходимости получать визу (если в поле указано число, «VF» или «VOA»):

```
[37]: # Задаём переменные:
      cntr = passportdata[2:end,1]
      vf = (x -> typeof(x)==Int64 || x == "VF" || x == "VOA" ? 1 : 0).(passportdata[2:end,2:end]);
```

Рис. 3.37: Задаём переменные

Определяем объект модели:

```
[38]: # Определение объекта модели с именем model:
      model = Model(GLPK.Optimizer)

[38]: A JuMP Model
      ├ solver: GLPK
      ├ objective_sense: FEASIBILITY_SENSE
      ├ num_variables: 0
      ├ num_constraints: 0
      └ Names registered in the model: none
```

Рис. 3.38: Определение объекта модели с именем model

Добавляем переменные, ограничения и целевую функцию:

```
[39]: # Переменные, ограничения и целевая функция:
      @variable(model, pass[1:length(cntr)], Bin)
      @constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
      @objective(model, Min, sum(pass))
```

[39]: $pass_1 + pass_2 + pass_3 + pass_4 + pass_5 + pass_6 + pass_7 + pass_8 + pass_9 + pass_{10} + pass_{11} + pass_{12} +$

Рис. 3.39: Переменные, ограничения и целевая функция

Для решения задачи оптимизации вызываем функцию оптимизации:

```
[40]: # Вызов функции оптимизации:
      JuMP.optimize!(model)
      termination_status(model)

[40]: OPTIMAL::TerminationStatusCode = 1
```

Рис. 3.40: Вызов функции оптимизации

Просматриваем результат:

```
[41]: # Просмотр результата:
print(JuMP.objective_value(model)," passports: ",join(cntr[findall(JuMP.value.(pass) .== 1)],", "))
```

```
34.0 passports: Afghanistan, Australia, Bahrain, Cameroon, Comoros, Congo, Djibouti, Dominican Republic, Eritrea, Guinea-Bissau, Hong Kong, Iran, Kenya, Kuwait, Liberia, Liby
a, Madagascar, Maldives, Mauritania, Morocco, Nepal, New Zealand, North Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Singapore, Somalia, South Sudan, Spain, Syria,
Turkmenistan, United States
```

Рис. 3.41: Просмотр результата

3.5 Портфельные инвестиции

Портфельные инвестиции — размещение капитала в ценные бумаги, формируемые в виде портфеля ценных бумаг, с целью получения прибыли. Инвестиционный портфель — процесс стратегического управления капиталом как оптимизированной, единой системой инвестиционных ценностей. Предположим требуется решить оптимизационную задачу в следующей формулировке. Имеется капитал в 1000 ден. ед., который планируется инвестировать в три компании — Microsoft, Facebook, Apple. При этом есть данные еженедельных значений цен на акции этих компаний за определённый период времени. Необходимо определить доходность акций каждой компании за рассматриваемый период времени, после чего инвестировать в эти три компании так, чтобы получить возврат в размере не менее 2% от вложенной суммы. Для решения оптимизационной задачи будем использовать пакет Convex.jl и оптимизатор (решатель) SCS. Кроме того, понадобится пакет Statistics.jl для получения матрицы рисков на основе расчёта ковариационных значений по доходности.

Сначала требуется подгрузить имеющиеся данные о еженедельных значениях цен на акции рассматриваемых компаний, отобразить данные на графике. Предположим данные размещены в файле stock_prices.xlsx в каталоге data в проекте. Подключаем пакеты:

```
*]: # Подключение необходимых пакетов:
import Pkg
Pkg.add("DataFrames")
Pkg.add("XLSX")
Pkg.add("Plots")
Pkg.add("PyPlot")
Pkg.add("Convex")
Pkg.add("SCS")
Pkg.add("Statistics")
using DataFrames
using XLSX
using Plots
pyplot()
using Convex
using SCS
using Statistics
```

```
   Resolving package versions...
     Project No packages added to or removed fro
    Manifest No packages added to or removed fro
```

Рис. 3.42: Подключение необходимых пакетов

Подгружаем данные еженедельных значений цен на акции компаний за определённый период времени во фрейм:

```
[45]: # Считываем данные и размещаем их во фрейм:
T = DataFrame(XLSX.readtable("stock_prices.xlsx","Sheet2"))
```

[45]: 13×3 DataFrame

Row	MSFT	FB	AAPL
	Any	Any	Any
1	101.93	137.95	148.26
2	102.8	143.8	152.29
3	107.71	150.04	156.82
4	107.17	149.01	157.76
5	102.78	165.71	166.52
6	105.67	167.33	170.41
7	108.22	162.5	170.42
8	110.97	161.89	172.97
9	112.53	162.28	174.97
10	110.51	169.6	172.91
11	115.91	165.98	186.12
12	117.05	164.34	191.05
13	117.94	166.69	189.95

Рис. 3.43: Считываем данные и размещаем их во фрейм

Отображаем данные на графике:

```
[46]: # Построение графика:
plot(T[!,:MSFT],label="Microsoft")
plot!(T[!,:AAPL],label="Apple")
plot!(T[!,:FB],label="FB")
```

[46]:

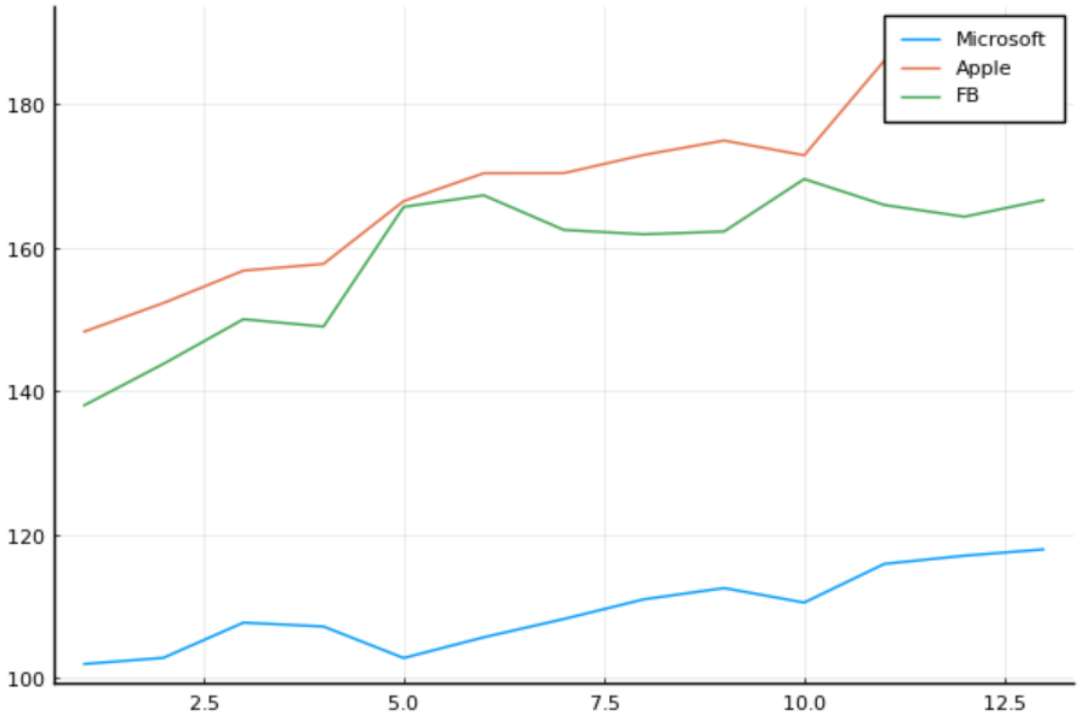

Рис. 3.44: Построение графика

Для дальнейших расчётов данные по ценам на акции переформируем из фрейма в матрицу:

```
[47]: # Данные о ценах на акции размещаем в матрице:
      prices_matrix = Matrix(T)
```

```
[47]: 13×3 Matrix{Any}:
       101.93  137.95  148.26
       102.8   143.8   152.29
       107.71  150.04  156.82
       107.17  149.01  157.76
       102.78  165.71  166.52
       105.67  167.33  170.41
       108.22  162.5   170.42
       110.97  161.89  172.97
       112.53  162.28  174.97
       110.51  169.6   172.91
       115.91  165.98  186.12
       117.05  164.34  191.05
       117.94  166.69  189.95
```

Рис. 3.45: Данные о ценах на акции размещаем в матрице

Вычисление матрицы доходности за период времени:

```
[48]: # Вычисление матрицы доходности за период времени:
      M1 = prices_matrix[1:end-1,:]
      M2 = prices_matrix[2:end,:]
      # Матрица доходности:
      R = (M2.-M1)./M1
```

```
[48]: 12×3 Matrix{Float64}:
        0.00853527   0.0424067    0.027182
        0.0477626    0.0433936    0.0297459
       -0.00501346  -0.00686484   0.00599413
       -0.040963     0.112073     0.0555274
        0.0281183    0.00977611   0.0233606
        0.0241317   -0.0288651    5.8682e-5
        0.0254112   -0.00375385   0.014963
        0.0140579    0.00240904   0.0115627
       -0.0179508    0.0451072   -0.0117734
        0.0488644   -0.0213443    0.0763981
        0.00983522  -0.00988071   0.0264883
        0.00760359   0.0142996   -0.00575766
```

Рис. 3.46: Вычисление матрицы доходности за период времени

Далее необходимо сформировать матрицу рисков — ковариационную матрицу рассчитанных цен доходности:

```
[49]: # Матрица рисков:
      risk_matrix = cov(R)

[49]: 3×3 Matrix{Float64}:
        0.000659383  -0.000630653  0.000139112
       -0.000630653   0.00152162   0.000192288
        0.000139112   0.000192288  0.000635503
```

Рис. 3.47: Матрица рисков

```
[50]: # Проверка положительной определённости матрицы рисков:
      isposdef(risk_matrix)

[50]: true
```

Рис. 3.48: Проверка положительной определённости матрицы рисков

Доход от каждой из компаний получим из матрицы доходности как вектор средних значений:

```
[51]: # Доход от каждой из компаний:
      r = mean(R,dims=1)[:]

[51]: 3-element Vector{Float64}:
       0.012532748705136572
       0.016563036855293173
       0.02114580465503291
```

Рис. 3.49: Доход от каждой из компаний

Формируем вектор инвестиций:

```
[52]: # Вектор инвестиций:
      x = Variable(length(r))

[52]: Variable
      size: (3, 1)
      sign: real
      vexity: affine
      id: 753…560
```

Рис. 3.50: Вектор инвестиций

Определяем объект модели в соответствии с формулировкой оптимизационной задачи (при этом делаем задачу совместимой с DCP в соответствии с требованием пакета Convex.jl — см. http://cvxr.com/cvx/doc/dcp.html)

```
[54]: # Объект модели:
      problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x>=0.02;x.>=0])
```

```
[54]: Problem statistics
        problem is DCP         : true
        number of variables    : 1 (3 scalar elements)
        number of constraints  : 5 (5 scalar elements)
        number of coefficients : 19
        number of atoms        : 14

      Solution summary
        termination status : OPTIMIZE_NOT_CALLED
        primal status      : NO_SOLUTION
        dual status        : NO_SOLUTION

      Expression graph
        minimize
         └─ * (convex; positive)
            ├─ [1;;]
            └─ qol_elem (convex; positive)
               ├─ norm2 (convex; positive)
               │  └─ …
               └─ [1.0;;]
        subject to
         ├─ == constraint (affine)
         │  └─ + (affine; real)
         │     ├─ sum (affine; real)
         │     │  └─ …
         │     └─ [-1;;]
         ├─ ≥ constraint (affine)
         │  └─ + (affine; real)
         │     ├─ * (affine; real)
         │     │  ├─ …
         │     │  └─ …
         │     └─ [-0.02;;]
         ├─ ≥ constraint (affine)
```

Рис. 3.51: Объект модели

Решаем поставленную задачу:

```
[55]: # Находим решение:
solve!(problem, SCS.Optimizer)
```

```
------------------------------------------------------------------
               SCS v3.2.11 - Splitting Conic Solver
        (c) Brendan O'Donoghue, Stanford University, 2012
------------------------------------------------------------------
problem:  variables n: 5, constraints m: 12
cones:    z: primal zero / dual free vars: 1
          l: linear vars: 4
          q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 22, nnz(P): 0
------------------------------------------------------------------
 iter | pri res | dua res |   gap   |   obj   |  scale  | time (s)
------------------------------------------------------------------
     0| 1.45e+01  1.00e+00  2.01e+01 -9.96e+00  1.00e-01  2.29e-03
    75| 1.70e-04  3.73e-05  5.25e-06  4.62e-04  1.00e-01  2.65e-03
------------------------------------------------------------------
status:  solved
timings: total: 2.65e-03s = setup: 1.18e-04s + solve: 2.54e-03s
         lin-sys: 1.65e-05s, cones: 1.43e-05s, accel: 5.70e-06s
------------------------------------------------------------------
objective = 0.000462
------------------------------------------------------------------
[ Info: [Convex.jl] Compilation finished: 7.32 seconds, 1007.784 MiB of memory allocated
```

```
[55]: Problem statistics
  problem is DCP         : true
  number of variables    : 1 (3 scalar elements)
  number of constraints  : 5 (5 scalar elements)
  number of coefficients : 19
  number of atoms        : 14

Solution summary
  termination status : OPTIMAL
  primal status      : FEASIBLE_POINT
  dual status        : FEASIBLE_POINT
  objective value    : 0.0005
```

Рис. 3.52: Находим решение

Выводим значения компонент вектора инвестиций:

```
[56]: x

[56]: Variable
      size: (3, 1)
      sign: real
      vexity: affine
      id: 753…560
      value: [0.07740709795031023, 0.11606771003983785, 0.8065276266877958]
```

Рис. 3.53: Результат

Проверяем выполнение условия $\sum_{i=1}^{3} x_i = 1$:

```
[57]: sum(x.value)

[57]: 1.000002434677944
```

Рис. 3.54: Проверка выполнения условия

Проверяем выполнение условия на уровень доходности от 2%:

```
[58]: r'*x.value

[58]: 1×1 adjoint(::Vector{Float64}) with eltype Float64:
       0.019947233108531883
```

Рис. 3.55: Проверка выполнения условия на уровень доходности от 2%

Переводим процентные значения компонент вектора инвестиций в фактические денежные единицы:

```
[59]: x.value .* 1000

[59]: 3×1 Matrix{Float64}:
        77.40709795031023
       116.06771003983785
       806.5276266877958
```

Рис. 3.56: Перевод процентных значений компонент вектора инвестиций в фактические денежные единицы

В результате, получаем: надо инвестировать 67.9 ден. ед. в Microsoft, 122.3 ден. ед. в Facebook, 809.7 ден. ед. в Apple.

3.6 Восстановление изображения

Предположим есть изображение, на котором были изменены некоторые пиксели. Требуется восстановить неизвестные пиксели путём решения задачи оптимизации. Будем использовать пакет Convex.jl и оптимизатор (решатель) SCS, пакет ImageMagick.jl для работы с изображениями:

[*]:
```
# Подключение необходимых пакетов:
import Pkg
Pkg.add("ImageMagick")
Pkg.add("Convex")
Pkg.add("SCS")
using Images
using Convex
using SCS
```

```
   Resolving package versions...
   Installed libzip_jll ─────── v1.11.3+0
   Installed FFTW_jll ──────── v3.3.11+0
   Installed LittleCMS_jll ─── v2.17.0+0
   Installed OpenJpeg_jll ──── v2.5.5+0
   Installed ImageMagick_jll ─ v7.1.2011+0
   Installed ImageMagick ────── v1.4.2
  Installing 5 artifacts
```

Рис. 3.57: Подключение необходимых пакетов

Загрузим изображение для последующей обработки:

```
[68]: # Считывание исходного изображения:
using Images
Kref = load("khiam-small.jpg")
```

[68]:

Рис. 3.58: Считывание исходного изображения

Преобразуем изображение в оттенки серого и испортим некоторые пиксели:

```
[69]: K = copy(Kref)
      p = prod(size(K))
      missingids = rand(1:p,400)
      K[missingids] .= RGBX{N0f8}(0.0,0.0,0.0)
      K
      Gray.(K)
```

[69]:

Рис. 3.59: Преобразование изображения в оттенки серого и порча пикселей

Формируем матрицу со значениями цветов:

```
[71]: # Матрица цветов:
      Y = Float64.(Gray.(K));
      Y
```

```
[71]: 283×283 Matrix{Float64}:
 0.101961   0.0627451  0.0784314  0.0941176  …  0.509804  0.552941  0.666667
 0.0666667  0.0980392  0.0745098  0.054902      0.505882  0.584314  0.501961
 0.0784314  0.0862745  0.0784314  0.0901961     0.6       0.701961  0.615686
 0.0862745  0.0666667  0.0745098  0.0941176     0.658824  0.705882  0.145098
 0.0784314  0.101961   0.0901961  0.0745098     0.713725  0.682353  0.231373
 0.0745098  0.0745098  0.0784314  0.0862745  …  0.729412  0.701961  0.168627
 0.12549    0.0980392  0.0862745  0.0862745     0.627451  0.466667  0.192157
 0.439216   0.447059   0.305882   0.137255      0.231373  0.184314  0.137255
 0.458824   0.454902   0.45098    0.454902      0.196078  0.101961  0.117647
 0.45098    0.466667   0.458824   0.45098       0.584314  0.121569  0.137255
 0.458824   0.458824   0.458824   0.454902   …  0.521569  0.513725  0.12549
 0.466667   0.45098    0.458824   0.47451       0.576471  0.741176  0.117647
 0.45098    0.45098    0.462745   0.458824      0.560784  0.67451   0.117647
 ⋮                                            ⋱  ⋮
 0.494118   0.47451    0.47451    0.462745      0.427451  0.435294  0.443137
 0.47451    0.482353   0.470588   0.470588      0.439216  0.431373  0.431373
 0.494118   0.501961   0.470588   0.45098       0.447059  0.447059  0.0
 0.470588   0.494118   0.490196   0.482353      0.431373  0.419608  0.419608
 0.458824   0.482353   0.47451    0.466667   …  0.454902  0.435294  0.423529
 0.443137   0.458824   0.45098    0.45098       0.309804  0.32549   0.34902
 0.47451    0.478431   0.462745   0.462745      0.341176  0.345098  0.360784
 0.482353   0.478431   0.458824   0.458824      0.423529  0.372549  0.321569
 0.552941   0.552941   0.541176   0.533333      0.447059  0.411765  0.372549
 0.552941   0.545098   0.576471   0.552941   …  0.435294  0.423529  0.407843
 0.564706   0.552941   0.54902    0.505882      0.439216  0.431373  0.419608
 0.568627   0.552941   0.517647   0.462745      0.439216  0.431373  0.427451
```

Рис. 3.60: Матрица цветов

Далее необходимо сформировать новую матрицу □, в которой минимизируется норма ядра матрицы (т.е. сумма сингулярных чисел элементов матрицы) так, что элементы, которые уже известны в матрице □, остаются теми же самыми в матрице □:

```
[72]: correctids = findall(Y[:].!=0)
      X = Convex.Variable(size(Y))
      problem = minimize(nuclearnorm(X))
      problem.constraints += X[correctids]==Y[correctids]
```

```
┌ Warning: Concatenating collections of constraints together with
s of constraints.
└ @ Convex C:\Users\eldar\.julia\packages\Convex\b2kc4\src\deprecat
```

```
[72]: 1-element Vector{Constraint}:
       == constraint (affine)
      └─ + (affine; real)
         ├─ index (affine; real)
         │  └─ 283×283 real variable (id: 184…418)
         └─ 79691×1 Matrix{Float64}
```

Рис. 3.61: Формирование матрицы с минимизированием нормы ядра матрицы

Решаем поставленную задачу:

```
[66]: # Находим решение:
      solve!(problem, SCS.Optimizer(eps=1e-3, alpha=1.5))
```

```
MethodError: no method matching SCS.Optimizer(; eps::Float64, alpha::Float64)
This method does not support all of the given keyword arguments (and may not support any).

Closest candidates are:
  SCS.Optimizer() got unsupported keyword arguments "eps", "alpha"
   @ SCS C:\Users\eldar\.julia\packages\SCS\iYFll\src\MOI_wrapper\MOI_wrapper.jl:163


Stacktrace:
 [1] top-level scope
   @ In[66]:2
 [2] eval(m::Module, e::Any)
   @ Core .\boot.jl:489
```

Рис. 3.62: Находим решение

3.7 Задания для самостоятельного выполнения

1.

```julia
using JuMP
using GLPK
# Создаём модель
model = Model(GLPK.Optimizer)
# Определяем переменные (с учётом ограничений x1 ≤ 10, x2 ≥ 0, x3 ≥ 0)
@variable(model, 0 <= x1 <= 10)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0)
# Задаём целевую функцию (максимизация)
@objective(model, Max, x1 + 2x2 + 5x3)
# Добавляем ограничения
@constraint(model, -x1 + x2 + 3x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10)
# Решаем задачу
optimize!(model)
# Выводим результаты
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Значения переменных: ")
println("x1 = ", value(x1))
println("x2 = ", value(x2))
println("x3 = ", value(x3))
```

```
Статус решения: OPTIMAL
Оптимальное значение целевой функции: 19.0625
Значения переменных:
x1 = 10.0
x2 = 2.1875
x3 = 0.9375
```

Рис. 3.63: Задание №1

2.

```julia
using JuMP
using GLPK
# Данные задачи
A = [1 1 9 5; 3 5 0 8; 2 0 6 13]
b = [7, 3, 5]
c = [1, 3, 5, 2]
# Создание модели
model = Model(GLPK.Optimizer)
# Объявление переменных (вектор x)
n = size(A, 2)
@variable(model, x[1:n] >= 0)
# Целевая функция (cᵀ * x)
@objective(model, Max, dot(c, x))
# Ограничения (Ax ≤ b)
@constraint(model, A * x .<= b)
# Решение задачи
optimize!(model)
# Вывод результатов
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Оптимальные значения переменных x: ", value.(x))
```

```
Оптимальное значение целевой функции: 5.355555555555556
Оптимальные значения переменных x: [0.0, 0.6, 0.7111111111111111, 0.0]
```

Рис. 3.64: Задание №2

3.

```julia
using Convex
using SCS
using Random
# Устанавливаем seed для воспроизводимости (опционально)
Random.seed!(123)
# Задаём размеры матрицы A (m × n)
m, n = 3, 4  # пример размеров, можно изменить
# Генерируем случайную матрицу A и вектор b
A = randn(m, n)  # случайная матрица m×n
b = randn(m)     # случайный вектор длины m
# Создаём переменную оптимизации x (вектор длины n)
x = Variable(n)
# Формулируем целевую функцию: ‖Ax − b‖₂²
objective = sumsquares(A * x - b)  # sumsquares вычисляет сумму квадратов элементов вектора
# Задаём ограничение: x ≥ 0
constraints = [x >= 0]
# Создаём задачу оптимизации
problem = minimize(objective, constraints)
# Решаем задачу с помощью SCS
solve!(problem, SCS.Optimizer)
# Выводим результаты
println("Оптимальное значение x: ", x.value)
println("Минимальное значение ‖Ax − b‖₂²: ", objective_value(model))(problem)
```

Рис. 3.65: Задание №3

```
------------------------------------------------------------------
Оптимальное значение x: [-2.5278850079144356e-5; 3.937682925084042; 1.2944095948975405; 4.543555600991451;;]
Минимальное значение ‖Ax − b‖₂²: 5.355555555555556
```

Рис. 3.66: Задание №3

4 Выводы

В результате выполнения лабораторной работы, я освоил пакеты Julia для решения задач оптимизации.